FermiWick

Reference manual

Every function, option and convention, with an example computed by the package

Nazzareno Africani · SISSA · version 0.3.0 · November 2026

Contents

1 Installation and loading 1

2 Conventions 1

3 Operators 2

4 Products and bases 4

5 Gaussian states 7

6 Wick's theorem and mean field 9

7 LaTeX 10

8 Gutzwiller projectors 11

9 Lattices 12

10 Matrices and exact diagonalisation 12

1 Installation and loading

FermiWick is a paclet. Install the paclet of the latest release once, then load it in every session, with the Gutzwiller and Fock modules when needed; the guided tour comes with the paclet:

```
PacletInstall["https://github.com/n4sd4q/FermiWick/releases/download/v0.3.0/FermiWick-0.3.0.paclet"]
Needs["FermiWick`"]; Needs["FermiWick`Gutzwiller`"]; Needs["FermiWick`Fock`"];
NotebookOpen[PacletObject["FermiWick"]["AssetLocation", "Examples"]]
```

To use a clone of the repository without installing, load its folder before `Needs`:

```
PacletDirectoryLoad["/path/to/FermiWick/FermiWick"]
```

FermiWick has no dependencies. It needs Mathematica 13.0 or later and is tested with 15.0.

Loaded in a notebook, the package prints a banner (the version, the license, how to cite it, links to the repository and the website) and looks for a newer release; a script prints nothing and never checks.

`FermiWickUpdateCheck, $FermiWickUpdateCheck`

At most once a day, in the background, loading the package in a notebook asks GitHub (`api.github.com`) for the latest release and, if it is newer than `$FermiWickVersion`, adds to the banner the `PacletInstall` line of that release; any failure is silent, and a session without access to the Internet never asks. `$FermiWickUpdateCheck = False` before `Needs["FermiWick`"]` (in `init.m` for good) turns the check off. `FermiWickUpdateCheck[]` asks by hand and gives the version loaded, the latest one and the line that installs it, or `$Failed` with a message.

The test suite runs with `math -script Tests/RunTests.wls`.

2 Conventions

- **Operators.** `c[l1, l2, ...]` and `cDag[l1, l2, ...]` annihilate and create a fermion in the mode with labels `l1, l2, ...`; `DeclareModeLabels` sets the meaning of each label. By default `c[i]` is a spinless fermion on site `i` and `c[i, s]` a fermion on site `i` with spin `s`. A site is any expression, and two different symbolic sites are different sites: `KroneckerDelta[i, j]` is 0. A spin is `SpinUp`, `SpinDown` or a symbol, split into ↑ and ↓ when needed. Generic labels (orbitals, momenta, ...) are compared with `KroneckerDelta`, which is kept and substituted in what it multiplies.
- **Products.** `a ** b` is the product in the given order; `a ⋅ b` (`\[CenterDot]`) is the product brought to normal order. Scalars (anything free of operators) factor out.
- **Normal order.** All $c^\dagger$ to the left of all c; results are expanded sums of monomials.
- **Contractions.** On a pair of sites, in spin channels: $\langle c^\dagger_{i\sigma} c_{j\sigma'}\rangle = \chi_{ij}\delta_{\sigma\sigma'} + \sum_a \chi^a_{ij}\sigma^a_{\sigma'\sigma}$ and $\langle c_{i\sigma} c_{j\sigma'}\rangle = [(\Delta_{ij} + \sum_a \Delta^a_{ij}\sigma^a)\epsilon]_{\sigma\sigma'}$, with $\epsilon_{\uparrow\downarrow} = 1$, $a = x, y, z$ (written `"x"`, `"y"`, `"z"`): `\[Chi][i, j]`, `\[Chi][i, j, a]`, `\[CapitalDelta][i, j]`, `\[CapitalDelta][i, j, a]`. Δ is symmetric and Δ^a antisymmetric in the sites; Hermiticity gives $\chi_{ji} = \chi^*_{ij}$, $\chi^a_{ji} = \chi^{a*}_{ij}$, and the amplitudes are written with the sites in canonical order (`Conjugate[\[Chi][1, 2]]` for χ_{21}). On one site χ_{ii} is the density per spin and $\chi^a_{ii} = \langle S^a_i\rangle$. In the general state by spin components, $\rho_{i\sigma j\sigma'} = \langle c^\dagger_{i\sigma} c_{j\sigma'}\rangle$ and $\kappa_{i\sigma j\sigma'} = \langle c_{i\sigma} c_{j\sigma'}\rangle$.
- **Complex conjugation.** The amplitudes are complex; every other symbol is taken as real by `Dagger`, unless declared complex.

Symbolic spins and sites:

```
NormalOrder[c[i, σ] ** cDag[j, σ]]
```

$$-\delta_{\downarrow,\sigma} c^\dagger_{j\downarrow} c_{i\downarrow} - \delta_{\uparrow,\sigma} c^\dagger_{j\uparrow} c_{i\uparrow}$$

```
NormalOrder[c[i, σ] ** cDag[i, σ]]
```

$$\delta_{\downarrow,\sigma}(1 - c^\dagger_{i\downarrow} c_{i\downarrow}) + \delta_{\uparrow,\sigma}(1 - c^\dagger_{i\uparrow} c_{i\uparrow})$$

3 Operators

`c, cDag`

The ladder operators. Their square vanishes:

```
NormalOrder[cDag[1, "↑"] ** cDag[1, "↑"]]
```

```
0
```

`DeclareModeLabels, $ModeLabels`

`DeclareModeLabels[{type1, type2, ...}]` declares the labels of the operators with that number of labels: `"Site"` (locality: products are reduced site by site), `"Spin"` (a spin $\frac{1}{2}$; the spin operators below need modes `c[i, s]`), or any other name, a generic label. Defaults: `{"Site"}`, `{"Site", "Spin"}`, and `{"Site", "Label", ...}` for more labels. `$ModeLabels` holds the declarations. Modes with no site are normal-ordered without the site-by-site reduction.

```
DeclareModeLabels[{"Site", "Orbital", "Spin"}];
NormalOrder[c[1, x, SpinUp] ** cDag[1, y, SpinUp] ** c[2, x, SpinDown]]
```

$$c_{2,x,\downarrow}\delta_{x,y} - c^\dagger_{1,y,\uparrow}c_{1,x,\uparrow}c_{2,x,\downarrow}$$

`SpinUp, SpinDown, $Spins`

`SpinUp` and `SpinDown` are the spins $\uparrow$ and $\downarrow$, `$Spins` their list, for spin sums:

```
NormalOrder[Sum[cDag[1, s] ** c[2, s], {s, $Spins}] ** c[3, SpinUp]]
```

$$c^\dagger_{1\downarrow}c_{2\downarrow}c_{3\uparrow} + c^\dagger_{1\uparrow}c_{2\uparrow}c_{3\uparrow}$$

`nOp, nTotOp, SzOp, SpOp, SmOp, DOp, B, Bdag`

The local operators of a site with spin; they stay compact, and `ToLadderBasis` writes them with c, $c^\dagger$. All the composite operators and their definitions:

FermiWick	operator	definition
`nOp[i, s]`	$n_{i\sigma}$	$c^\dagger_{i\sigma}c_{i\sigma}$
`nTotOp[i]`, `nOp[i]`	n_i	$n_{i\uparrow} + n_{i\downarrow}$
`SzOp[i]`	S^z_i	$\frac{1}{2}\left(n_{i\uparrow} - n_{i\downarrow}\right)$
`SpOp[i]`, `SmOp[i]`	S^+_i, S^-_i	$c^\dagger_{i\uparrow}c_{i\downarrow}$, $c^\dagger_{i\downarrow}c_{i\uparrow}$
`SpinOperator[i, a]`	S^a_i	$\frac{1}{2}\sum_{\sigma\sigma'} c^\dagger_{i\sigma}\,\sigma^a_{\sigma\sigma'}\,c_{i\sigma'}$, $a = 1,2,3$
`DOp[i]`	D_i (double occupancy)	$n_{i\uparrow}n_{i\downarrow}$
`Bdag[i]`, `B[i]`	$B^\dagger_i$, B_i (doublon)	$c^\dagger_{i\uparrow}c^\dagger_{i\downarrow}$, $c_{i\downarrow}c_{i\uparrow}$
`EmptyProjector[i]`	P^0_i	$(1-n_{i\uparrow})(1-n_{i\downarrow}) = 1 - n_i + D_i$
`SingleOccupationProjector[i]`	P^1_i	$n_i - 2D_i$
`GutzwillerProjector[i]`	P^G_i	$\sqrt{2}\left[\cos\theta\, P^1_i + \sin\theta\,(P^0_i + D_i)\right]$
`SdotS[i, j]`	$\mathbf{S}_i \cdot \mathbf{S}_j$	$S^z_iS^z_j + \frac{1}{2}(S^+_iS^-_j + S^-_iS^+_j)$
`SdotCrossS[i, j, k]`	$\mathbf{S}_i \cdot (\mathbf{S}_j \times \mathbf{S}_k)$	$\sum_{abc}\epsilon_{abc}\,S^a_iS^b_jS^c_k$

Products of these on one site reduce with the algebra of the fermions (e.g. $D_in_i = 2D_i$, $(S^z_i)^2 = \frac{1}{4}n_i - \frac{1}{2}D_i$); operators on several sites and products with symbolic sites stay as products.

`GutzwillerProjector` is in `FermiWick`Gutzwiller`; one-site operators on several sites are products, `DOp[i, j]` $= D_i D_j$.

```
ToLadderBasis[SzOp[1]]
```

$$\frac{1}{2}c^\dagger_{1\uparrow}c_{1\uparrow} - \frac{1}{2}c^\dagger_{1\downarrow}c_{1\downarrow}$$

`SpinOperator, SdotS, SdotScrossS`

`SpinOperator[i, a]` is S^a_i, $a = 1,2,3$; `SdotS[i, j]` is $\mathbf{S}_i \cdot \mathbf{S}_j$ and `SdotScrossS[i, j, k]` the scalar chirality $\mathbf{S}_i \cdot (\mathbf{S}_j \times \mathbf{S}_k)$.

```
NormalOrder[Sum[SpinOperator[1, a] · SpinOperator[2, a], {a, 3}] - SdotS[1, 2]]
```

$$0$$

`EmptyProjector, SingleOccupationProjector`

$P^0_i = 1 - n_i + D_i$ and $P^1_i = n_i - 2D_i$, so that $P^0 + P^1 + D = 1$; with several sites they are products.

```
ToSpinChargeBasis[EmptyProjector[1] + SingleOccupationProjector[1] + DOp[1]]
```

$$1$$

`DefineOperator`

`DefineOperator[head, n, definition, display]` defines an operator on n sites: `definition` is a function of the sites written with known operators, `display` a function of the sites that gives its form on screen. The operator stays compact; `NormalOrder`, `WickVEV` and `MeanField` expand it, and `ToSpinChargeBasis` recognizes it again. Options: `"Recognize"` (default `True` for $n \geq 2$) and `"Required" -> All` (recognize it only when it is present whole; by default its diagonal part may leave a remainder, as in $J\,\mathbf{S}\cdot\mathbf{S} + (J_z - J)S^zS^z$). A one-site operator on several sites is the product. `$DefinedOperators` is the registry.

```
DefineOperator[Hop, 2, (cDag[#1, "↑"] ** c[#2, "↑"] + cDag[#1, "↓"] ** c[#2, "↓"]) &,
  Subscript["T", Row[{#1, #2}]] &];
ToSpinChargeBasis[NormalOrder[2 Hop[1, 2] ** nTotOp[3]]]
```

$$2\mathrm{T}_{12}n_3$$

`AdjointOperator`

The adjoint of a defined operator stays compact: `Dagger` gives the operator itself when it is Hermitian, the operator with its sites permuted when that is its adjoint (a hopping), and otherwise `AdjointOperator[head][sites]`, registered automatically, shown with a dagger, expanded and recognized like the operator itself. Here the hopping of the example above and an operator that annihilates a singlet pair on a bond:

```
DefineOperator[Pair, 2,
  (c[#1, "↑"] ** c[#2, "↓"] - c[#1, "↓"] ** c[#2, "↑"])/Sqrt[2] &,
  Subscript["b", Row[{#1, #2}]] &];
{Dagger[Hop[1, 2]], Dagger[Pair[1, 2]],
  ToSpinChargeBasis[Dagger[Pair[1, 2]] ** Pair[1, 2]]}
```

$$\left\{\mathrm{T}_{21},\ \mathrm{b}_{12}{}^{\dagger},\ \frac{1}{4}n_1 n_2 - \mathbf{S}_1\cdot\mathbf{S}_2\right\}$$

4 Products and bases

`NormalOrder`

Brings an expression to normal order. Products are separated site by site: each factor acts on one site and is even or odd, so the sites are reordered with the fermion sign only and each site is reduced once.

`CenterDot`

`a · b · ...` is the normal-ordered product.

```
ToSpinChargeBasis[SzOp[1] · SzOp[1]]
```

$$\frac{n_1}{4} - \frac{D_1}{2}$$

`OperatorCommutator, OperatorAnticommutator`

$ab - ba$ and $ab + ba$, in normal order.

```
OperatorAnticommutator[c[1, "↑"], cDag[1, "↑"]]
```

$$1$$

`ToSpinChargeBasis`

Writes an expression with n, S^z, $S^{\pm}$, D, B, $B^\dagger$ and recognizes the defined operators; `"Recognize" -> False` leaves them expanded.

```
ToSpinChargeBasis[cDag[1, "↑"] ** c[1, "↓"] ** cDag[2, "↓"] ** c[2, "↑"]]
```

$$S_1^+ S_2^-$$

```
ToSpinChargeBasis[SdotS[1, 2], "Recognize" -> False]
```

$$\frac{1}{2}S_1^- S_2^+ + \frac{1}{2}S_1^+ S_2^- + S_1^z S_2^z$$

`ToLadderBasis`

Writes the composite and the defined operators with c and $c^\dagger$.

`GroupLocalOperators`

Normal-orders and groups the ladder operators site by site, with the fermion sign.

```
GroupLocalOperators[cDag[1, "↑"] ** cDag[2, "↑"] ** c[2, "↑"] ** c[1, "↑"]]
```

$$c^\dagger_{1\uparrow} c_{1\uparrow} c^\dagger_{2\uparrow} c_{2\uparrow}$$

`OperatorCoefficients`

The association monomial → coefficient of an expression in normal order; the constant has the key 1. `OperatorCoefficients[expr, m]` is the coefficient of one monomial, written in any order. It reads the fields of a mean-field Hamiltonian.

```
OperatorCoefficients[2 x cDag[1, "↑"] ** c[2, "↑"] + y + c[2, "↑"] ** cDag[1, "↑"]]
```

```
<|1 -> y, cDag[1, "↑"]**c[2, "↑"] -> -1 + 2*x|>
```

```
OperatorCoefficients[2 x cDag[1, SpinUp] ** c[2, SpinUp], c[2, SpinUp] ** cDag[1,
    SpinUp]]
```

$$-2x$$

`Dagger`

The Hermitian conjugate: products reversed, $c \leftrightarrow c^\dagger$, $S^+ \leftrightarrow S^-$, $B \leftrightarrow B^\dagger$, coefficients conjugated. The amplitudes χ, Δ, ρ, κ are complex, the other symbols real; option `"Complex" -> {...}` declares more complex symbols. Operators defined with `DefineOperator` stay compact (`AdjointOperator`).

```
Dagger[t E^(I \[Alpha]) cDag[1, SpinUp] ** c[2, SpinUp] + χ[1, 2] SpOp[1]]
```

$$S_1^- \chi_{12}^* + e^{-i\alpha} t c_{2\uparrow}^\dagger c_{1\uparrow}$$

`PlusHC, HermitianOperatorQ`

`PlusHC[expr]` is `expr` + `expr`†; `HermitianOperatorQ[expr]` checks `expr` = `expr`†. Options as `Dagger`.

```
PlusHC[g c[1, SpinUp] ** c[2, SpinDown], "Complex" -> {g}]
```

$$g c_{1\uparrow} c_{2\downarrow} + g^* c_{2\downarrow}^\dagger c_{1\uparrow}^\dagger$$

`CleanDeltas, ResetNormalOrder, $SpinSymbols`

`CleanDeltas` simplifies the `KroneckerDelta` of sites and spins and does the sums over the spin; `ResetNormalOrder[]` clears the caches; `$SpinSymbols` lists the symbols used as spins.

5 Gaussian states

A Gaussian state is an object that gives the contraction $\langle O_1 O_2 \rangle$ of two ladder operators. It is declared by its symmetries, or by rules for the contractions. `state["Symmetries"]`, `state["Description"]` and `state["Contraction"]` give its parts.

`GaussianState`

Gaussian states, in which every expectation value follows from the contractions of two operators, are also called quasi-free states (pure, as Slater or BCS states, or mixed, as Gibbs states of quadratic Hamiltonians). `GaussianState[symmetries, options]`: the state with the given symmetries, written in spin channels (Section 2). `GaussianState[]` is the general state. Each symmetry sets channels to zero or makes them real:

symmetry	what it leaves
`"SpinRotation"`	χ, Δ
`"SpinRotationZ"`	χ, χ^z, Δ, Δ^z
`"Charge"`	χ, χ^a (no pairing)
`"TimeReversal"`	χ, Δ real; χ^a, Δ^a imaginary; no magnetization

Options:

option	default	
`"Hopping"`, `"Pairing"`	χ, Δ	names of the amplitudes
`"Filling"`	`Automatic`	χ_{ii}; `Automatic`: the symbol χ_{ii}
`"Magnetization"`	`Automatic`	$\langle S_i^z \rangle$, a vector, or a function of the site
`"OnSitePairing"`	`Automatic`	Δ_{ii}
`"Hermitian"`	`True`	$\chi_{ji} = \chi_{ij}^*$; `False`: χ_{ji} is a symbol of its own
`"Rules"`	`{}`	replacements applied to the results

`GaussianState[{{op1, op2} :> value, ...}]` declares the contractions by rules instead, with patterns in the operators. Pairs with no rule contract to zero, and $\langle c\, c^\dagger \rangle$ follows from $\langle c^\dagger c \rangle$ by the anticommutator when it has no rule of its own. Two sites with hopping g:

```
dimer = GaussianState[{{cDag[i_, s_], c[j_, t_]} :> KroneckerDelta[s, t] If[i === j,
    1/2, g]}];
WickVEV[nOp[1, "↑"] ** nOp[2, "↑"], dimer]
```

$$\frac{1}{4} - g^2$$

ContractionTable

`ContractionTable[state, {i, j, ...}]` shows all the contractions between the modes of the sites, $\langle c_a^\dagger c_b \rangle$ and $\langle c_a c_b \rangle$; with a third argument `"Normal"` or `"Anomalous"` it gives the matrix, with the modes in the order of `Tuples[{sites, $Spins}]`. The general state on one site, and the symmetric one:

```
MatrixForm[ContractionTable[GaussianState[], {1}, "Normal"]]
```

$$\begin{pmatrix} \chi_{11} + \chi_{11}^z & \chi_{11}^x + i\chi_{11}^y \\ \chi_{11}^x - i\chi_{11}^y & \chi_{11} - \chi_{11}^z \end{pmatrix}$$

```
MatrixForm[ContractionTable[GaussianState[{"SpinRotation", "Charge"}, "Filling" -> ν
    ], {1, 2}, "Normal"]]
```

$$\begin{pmatrix} \nu & 0 & \chi_{12} & 0 \\ 0 & \nu & 0 & \chi_{12} \\ \chi_{12}^* & 0 & \nu & 0 \\ 0 & \chi_{12}^* & 0 & \nu \end{pmatrix}$$

Contraction

`Contraction[state, op1, op2]`; `Contraction[op1, op2]` uses `$State`.

```
{Contraction[dimer, cDag[1, "↑"], c[2, "↑"]], Contraction[dimer, c[1, "↑"], cDag[2,
    "↑"]]}
```

$$\{g, -g\}$$

SpinSingletState

The usual shortcut: `GaussianState[{"SpinRotation", "Charge"}, "Filling" -> 1/2]`, or, with `"Pairing" -> \[CapitalDelta]`, the SU(2) state with singlet pairing and on-site pairing `"OnSitePairing"` (default 0). Options `"Hopping"`, `"Filling"`, `"Pairing"`, `"OnSitePairing"`, `"Hermitian"`.

```
WickVEV[DOp[1], SpinSingletState["Filling" -> ν, "Pairing" -> Δ, "OnSitePairing" -> η
    ]]
```

$$\eta\eta^* + \nu^2$$

`GeneralState`

The general state written by spin components instead of channels: ρ (option `"Normal"`, Hermitian) and, with `"Anomalous" -> \[Kappa]`, κ, antisymmetric and written in canonical order.

```
WickVEV[SzOp[1], GeneralState[]]
```

$$\frac{1}{2}\rho_{1,\uparrow,1,\uparrow} - \frac{1}{2}\rho_{1,\downarrow,1,\downarrow}$$

```
WickVEV[Bdag[1] ** B[2], GeneralState["Anomalous" -> κ]]
```

$$\kappa_{2,\downarrow,2,\uparrow}\kappa^*_{1,\downarrow,1,\uparrow} - \rho_{1,\downarrow,2,\uparrow}\rho_{1,\uparrow,2,\downarrow} + \rho_{1,\downarrow,2,\downarrow}\rho_{1,\uparrow,2,\uparrow}$$

`StateSelect, LocalPairQ, AnomalousPairQ`

`StateSelect[state, crit]` keeps the contractions of the pairs with `crit[op1, op2]` true. It is a filter, not a symmetry: it gives a physical state when the criterion is a symmetry or a decoupling of the sites (`LocalPairQ`: uncorrelated sites), and otherwise a way to isolate a class of terms. `AnomalousPairQ` is true for two c or two $c^\dagger$.

```
WickVEV[nTotOp[1] ** nTotOp[2], StateSelect[SpinSingletState[], LocalPairQ]]
```

$$1$$

`$State`

The state used when none is given, `SpinSingletState[]` by default.

6 Wick's theorem and mean field

`WickVEV`

`WickVEV[expr, state]` is the expectation value by Wick's theorem: the expression is normal-ordered, and each monomial is the sum over its pairings with the sign of the permutation. Options: `"MaxOrder" -> m`, at most m contractions between different sites; `"NormalOrder" -> False`, Wick on the expression as it is.

```
WickVEV[SdotS[1, 2] ** SdotS[2, 3], "MaxOrder" -> 2]
```

$$-\frac{3}{16}\chi_{13}\chi^*_{13}$$

`VerifyNormalOrder`

0 when `WickVEV` with and without normal ordering agree.

```
VerifyNormalOrder[c[1, "↑"] ** cDag[2, "↑"] ** c[2, "↑"] ** cDag[1, "↑"]]
```

$$0$$

`MeanField`

`MeanField[expr, state]` is the Hartree–Fock–Bogoliubov Hamiltonian of `expr`: every normal-ordered product $O_1 \cdots O_{2n}$ becomes

$$\sum_{p<q} (-1)^{p+q+1} O_p O_q \left\langle \prod_{r \neq p,q} O_r \right\rangle - (n-1) \langle O_1 \cdots O_{2n} \rangle,$$

two operators kept out in every way and the rest replaced by its expectation value. The result is quadratic, has the same expectation value as `expr` and the same first variation with respect to the state: it is the Hamiltonian whose Gibbs state makes $\langle H \rangle - TS$ stationary among Gaussian states. Quadratic terms are unchanged; odd terms give zero. The density–density interaction (Hartree and Fock):

```
MeanField[nTotOp[1] ** nTotOp[2]]
```

$$-\chi_{12}^* c_{1\downarrow}^\dagger c_{2\downarrow} - \chi_{12}^* c_{1\uparrow}^\dagger c_{2\uparrow} - \chi_{12} c_{2\downarrow}^\dagger c_{1\downarrow} - \chi_{12} c_{2\uparrow}^\dagger c_{1\uparrow} + c_{1\downarrow}^\dagger c_{1\downarrow} + c_{2\downarrow}^\dagger c_{2\downarrow} + c_{1\uparrow}^\dagger c_{1\uparrow} + c_{2\uparrow}^\dagger c_{2\uparrow} + 2\chi_{12}\chi_{12}^* - 1$$

```
MeanField[cDag[1, "↑"] ** c[2, "↑"]]
```

$$c_{1\uparrow}^\dagger c_{2\uparrow}$$

7 LaTeX

`ToTeX`

The LaTeX of an expression: ladder and local operators, spin operators, amplitudes and spins; the operators defined with `DefineOperator` are written with their display.

```
ToTeX[SdotS[1, 2] - 2 DOp[1] + χ[1, 2] cDag[1, SpinUp] ** c[2, SpinUp]]
```

```
\chi_{12} c^\dagger_{1\uparrow}c_{2\uparrow}+\mathbf S_{1}\cdot\mathbf S_{2}-2
    D_{1}
```

`WickPairings`

`WickPairings[product, state]` lists the pairings of a product of ladder operators that do not vanish in the state, in Wick's theorem applied to the product as written: `"Sign"`, `"Pairs"` (positions in the product) and `"Value"` (the product of the contractions).

```
WickPairings[cDag[1, SpinUp] ** c[2, SpinUp] ** cDag[2, SpinUp] ** c[1, SpinUp]]
```

```
{<|"Sign" -> 1, "Pairs" -> {{1, 2}, {3, 4}}, "Value" -> Conjugate[χ[1, 2]]*χ[1,
    2]|>,
 <|"Sign" -> 1, "Pairs" -> {{1, 4}, {2, 3}}, "Value" -> 1/4|>}
```

`WickTeX`

`WickTeX[product, state]` writes Wick's theorem for a product in LaTeX: the expectation value, the sum of the pairings drawn as contractions, their values and the result. Options `"Values"` and `"Result"` (default `True`) leave out the last two. The contractions use `simpler-wick`: `\usepackage{simpler-wick}` in the preamble, and two LaTeX runs. Overlapping contractions are drawn at different heights.

```
WickTeX[cDag[1, SpinUp] ** c[2, SpinUp] ** cDag[2, SpinUp] ** c[1, SpinUp], "Values"
    -> False]
```

```
\langle c^\dagger_{1\uparrow} c_{2\uparrow} c^\dagger_{2\uparrow}
    c_{1\uparrow}\rangle = \wick{\c1 c^\dagger_{1\uparrow} \c1 c_{2\uparrow} \c1
    c^\dagger_{2\uparrow} \c1 c_{1\uparrow}} + \wick{\c2 c^\dagger_{1\uparrow} \c1
    c_{2\uparrow} \c1 c^\dagger_{2\uparrow} \c2 c_{1\uparrow}} = \chi_{12}
    \chi_{12}^*+\frac{1}{4}
```

$$\langle c^\dagger_{1\uparrow} c_{2\uparrow} c^\dagger_{2\uparrow} c_{1\uparrow}\rangle = \overbrace{c^\dagger_{1\uparrow} c_{2\uparrow}}\,\overbrace{c^\dagger_{2\uparrow} c_{1\uparrow}} + \overbrace{c^\dagger_{1\uparrow} c_{2\uparrow} c^\dagger_{2\uparrow} c_{1\uparrow}} = \chi_{12}\chi_{12}^* + \frac{1}{4}$$

8 Gutzwiller projectors

Loaded by `Needs["FermiWick`Gutzwiller`"]`.

`GutzwillerProjector, \[Theta]`

$P_i^G = \sqrt{2}\,[\cos\theta\, P_i^1 + \sin\theta\,(P_i^0 + D_i)]$, the half-filling Gutzwiller operator with the variational angle θ; $\theta = 0$ projects on single occupation.

```
ToSpinChargeBasis[GutzwillerProjector[1]] /. θ -> 0
```

$$\sqrt{2}n_1 - 2\sqrt{2}D_1$$

`LocalSandwich`

`LocalSandwich[P, {i, j, ...}, X]` is $P_i P_j \cdots X \cdots P_j P_i$, normal-ordered; `LocalSandwich[PL, sites, X, PR]` uses different operators on the two sides.

```
ToSpinChargeBasis[LocalSandwich[GutzwillerProjector, {1}, DOp[1]]]
```

$$2D_1 \sin^2(\theta)$$

`GutzwillerVEV, GutzwillerSimplify`

`GutzwillerVEV[expr, state]` is `WickVEV` to first order in the contractions between different sites (the hopping amplitudes of the state), simplified by `GutzwillerSimplify`.

```
GutzwillerVEV[LocalSandwich[GutzwillerProjector, {1}, nTotOp[1]]]
```

$$1$$

9 Lattices

`DefinePeierlsHopping, DefineFlux`

`DefinePeierlsHopping[t, \[CurlyPhi]]` makes $t_{ab} = t\,e^{i\varphi_{ab}}$ with $\varphi_{ba} = -\varphi_{ab}$; `DefineFlux[\[CurlyPhi], {i, j, k, ...}, \[CapitalPhi]]` fixes the sum of the phases around the oriented loop to Φ, eliminating one free phase.

```
DefinePeierlsHopping[t, φ];
DefineFlux[φ, {1, 2, 3}, Φ];
Simplify[Subscript[t, 1, 2] Subscript[t, 2, 3] Subscript[t, 3, 1]]
```

$$t^3 e^{i\Phi}$$

10 Matrices and exact diagonalisation

Loaded by `Needs["FermiWick`Fock`"]`, in a cell of its own (the names must be known before the code that uses them is read):

```
Needs["FermiWick`Fock`"]
```

Conventions. The modes of a space are ladder operators of FermiWick, numbered site by site, then orbital by orbital, the spin fastest: $\alpha = (\text{site}-1)\,n_o n_s + (\text{orbital}-1)\,n_s + \text{spin}$, the order of `Tuples[{sites, orbitals, spins}]` and of `ContractionTable`. A basis state is the string of the occupations $(n_1,\dots,n_M)$, with index $k = \sum_\alpha n_\alpha 2^{M-\alpha}$ (the first mode is the most significant bit): the order of `Tuples[{0, 1}, M]`. The Jordan–Wigner sign is $c_\alpha|n\rangle = (-1)^{\sum_{\beta<\alpha} n_\beta}|n-e_\alpha\rangle$. The matrices are `SparseArray` objects; inside a sector the basis states are in increasing order of k.

`FockSpace, FockSpaceQ`

`FockSpace[sites]` is the space of spin-$\frac{1}{2}$ fermions on the sites, with the modes `c[i, s]`; `FockSpace[sites, "Orbitals" -> {o1, ...}]` has the modes `c[i, o, s]`, and `"Spins" -> None` gives spinless modes `c[i]` (or `c[i, o]`); `"Spins" -> {s1, s2}` names the two spins, the first one up. `space["Modes"]`, `space["Count"]` and `space["Dimension"]` (2^M) are its properties.

```
FockSpace[{1, 2}, "Orbitals" -> {"s", "p"}]["Modes"]
```

```
{c[1, "s", "↑"], c[1, "s", "↓"], c[1, "p", "↑"], c[1, "p", "↓"], c[2, "s", "↑"],
    c[2, "s", "↓"],
 c[2, "p", "↑"], c[2, "p", "↓"]}
```

`ModeIndex`

`ModeIndex[space, c[l1, l2, ...]]` (or `cDag`) is the index of a mode; `ModeIndex[space, l1, l2, ...]` the same from the labels. A mode outside the space gives a message and `$Failed`.

```
{ModeIndex[FockSpace[{1, 2}], c[2, "↑"]], FockSpace[Range[3], "Spins" ->
    None]["Modes"]}
```

```
{3, {c[1], c[2], c[3]}}
```

FockSector, FockSectorQ

`FockSector[space, "N" -> n, "Sz" -> sz, "Counts" -> {{modes, n}, ...}]` keeps the basis states with n particles, the given S^z (two spins) and n particles in each set of modes (ladder operators or indices). Every operator is built directly in a sector, without the whole space: sectors with fixed N are enumerated bit by bit and indexed through the combinatorial number system, so that time and memory scale with their dimension; sectors without fixed N need the whole space and are allowed up to `$FockFullLimit` modes (24). `ClearFockCache[]` forgets the sectors built so far.

```
{FockSector[FockSpace[Range[6]], "N" -> 6, "Sz" -> 0]["Dimension"],
 FockSector[FockSpace[{1, 2}], "Counts" -> {{{c[1, "↑"], c[1, "↓"]},
    2}}]["Dimension"]}
```

```
{400, 4}
```

FockMatrix

`FockMatrix[expr, target]` is the matrix of an expression of FermiWick: ladder operators, local and composite operators, operators defined with `DefineOperator` (also `GutzwillerProjector`), products with ** in the order written (no normal order is needed), coefficients numbers or symbols. `target` is a space, a sector (the image sector is found from the change of N, S^z and of every count) or a pair `{fromSector, toSector}`. The labels of every operator must be those of modes of the space; a symbolic spin must be summed first (`Sum[..., {s, $Spins}]`).

```
Normal[FockMatrix[cDag[1, "↑"] ** c[2, "↑"], FockSector[FockSpace[{1, 2}], "N" -> 1]]]
```

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

```
Normal[FockMatrix[J SdotS[1, 2], FockSector[FockSpace[{1, 2}], "N" -> 2, "Sz" -> 0]]]
```

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & -\frac{J}{4} & \frac{J}{2} & 0 \\ 0 & \frac{J}{2} & -\frac{J}{4} & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

FockExpectation

`FockExpectation[expr, v, sector]` is $\langle v|$ `expr` $|v\rangle$ for a normalized vector `v` of the sector (or space); an operator that leaves the sector gives 0. Here the singlet of two sites:

```
sec = FockSector[FockSpace[{1, 2}], "N" -> 2, "Sz" -> 0];
singlet = {0, 1, -1, 0}/Sqrt[2];
{FockExpectation[SdotS[1, 2], singlet, sec], FockExpectation[DOp[1], singlet, sec]}
```

$$\left\{-\frac{3}{4}, 0\right\}$$

StateToKet

`StateToKet[target, v]` writes a vector as kets of the local occupations (0, ↑, ↓, ↑↓ for spin $\frac{1}{2}$; the occupations of the modes otherwise), site by site.

```
StateToKet[sec, singlet]
```

```
Ket[{{"↓"}, {"↑"}}]/Sqrt[2] - Ket[{{"↑"}, {"↓"}}]/Sqrt[2]
```

OccupationBasis, BasisState

`OccupationBasis[target]` lists the occupation vectors of the basis states, in order; `BasisState[target, occupations]` is the basis vector with the given occupations.

```
{OccupationBasis[sec], Normal[BasisState[sec, {1, 0, 0, 1}]]}
```

```
{{{0, 0, 1, 1}, {0, 1, 1, 0}, {1, 0, 0, 1}, {1, 1, 0, 0}}, {0, 0, 1, 0}}
```

SpinSquared

`SpinSquared[target]` is $\mathbf{S}^2$ of the whole space or of a sector (two spins), built from S^+ and S^z: faster than the matrix of $\sum_{ij} \mathbf{S}_i \cdot \mathbf{S}_j$.

```
Eigenvalues[Normal[SpinSquared[sec]]]
```

```
{2, 0, 0, 0}
```

LowestStates

`LowestStates[H, k]` gives `{energies, vectors}` of the k lowest eigenstates of a Hermitian matrix: dense up to dimension `$FockDenseLimit` (3000), Arnoldi above, from several starting vectors when $k > 1$, so that degenerate multiplets are complete.

```
dimer = -Sum[cDag[1, s] ** c[2, s] + cDag[2, s] ** c[1, s], {s, $Spins}] + 4 (DOp[1]
    + DOp[2]);
N[Round[10^4 First[LowestStates[FockMatrix[dimer, sec], 2]]]/10^4]
```

```
{-0.8284, 0.}
```

LehmannGreen, GreenMatrix

LehmannGreen[H, space, modes] is the single-particle Green's function at zero temperature, $G_{ab}(z) = \langle c_a (z - H + E_0)^{-1} c_b^\dagger \rangle + \langle c_b^\dagger (z + H - E_0)^{-1} c_a \rangle$, of a Hamiltonian that conserves N (a matrix on the whole space, or a function sector → matrix), for the modes given (ladder operators or indices), averaged over the degenerate ground multiplet: an association with "GroundEnergy", "GroundSectors", "Degeneracy", "Poles" and "Residues". Options: "N" (the particle numbers where the ground state is sought, All), "UseSz" (sectors of S^z when H conserves it, Automatic), "Tolerance" (10^{-9}, the width of the ground multiplet), "ResidueCutoff" (10^{-12}). The residues add up to the identity (the last entry below). GreenMatrix[g, z] evaluates it at a complex frequency.

```
space = FockSpace[{1, 2}];
g = LehmannGreen[FockMatrix[dimer, space], space, {c[1, "↑"]}, "N" -> 2];
{N[Round[10^4 g["GroundEnergy"]]/10^4], g["Degeneracy"], Max[Abs[Total[g["Residues"]]
    - IdentityMatrix[1]]] < 10^-12}
```

```
{-0.8284, 1, True}
```

```
N[Round[10^4 GreenMatrix[g, 0.5 I]]/10^4]
```

```
{{-0.3469 - 0.7892*I}}
```

ParticleHoleOperator, TimeReversal

ParticleHoleOperator[space, first, last] is the unitary P with $P c_\alpha P^{-1} = c_\alpha^\dagger$ for the modes first..last (all by default): σ^x on each of them times $e^{-i\pi \sum_\alpha (\alpha-1) n_\alpha}$. TimeReversal[space] gives <|"U" -> U, "Apply" -> f, "Inverse" -> finv|> for the antiunitary $T = UK$ of spin-$\frac{1}{2}$ fermions, with f[v] = U.Conjugate[v].

```
With[{ph = ParticleHoleOperator[space]}, Max[Abs[ph . FockMatrix[c[1, "↑"], space] .
    Transpose[ph] - FockMatrix[cDag[1, "↑"], space]]]]
```

```
0
```

SectorBlock

SectorBlock[H, sector] is the block of a matrix on the whole space that acts in the sector.

```
SectorBlock[FockMatrix[DOp[1] + DOp[2], space], sec] == FockMatrix[DOp[1] + DOp[2],
    sec]
```

```
True
```